Emotion Radar

Extract emotion from text with Machine Learning

DM1590 - Machine Learning for Media Technology

Group 15: Fuad Mammadov, Qingya Li, Xinyutong Zhang, Jintong Jiang

Project Overview

Using a text dataset from Google's project "GoEmotion", we want to train a ML model that can detect one of the seven emotions from the given text.

01 Data Preparation

- goEmotion dataset
- data cleaning
- text to vector

02 Unsupervised: Clustering & Visualization

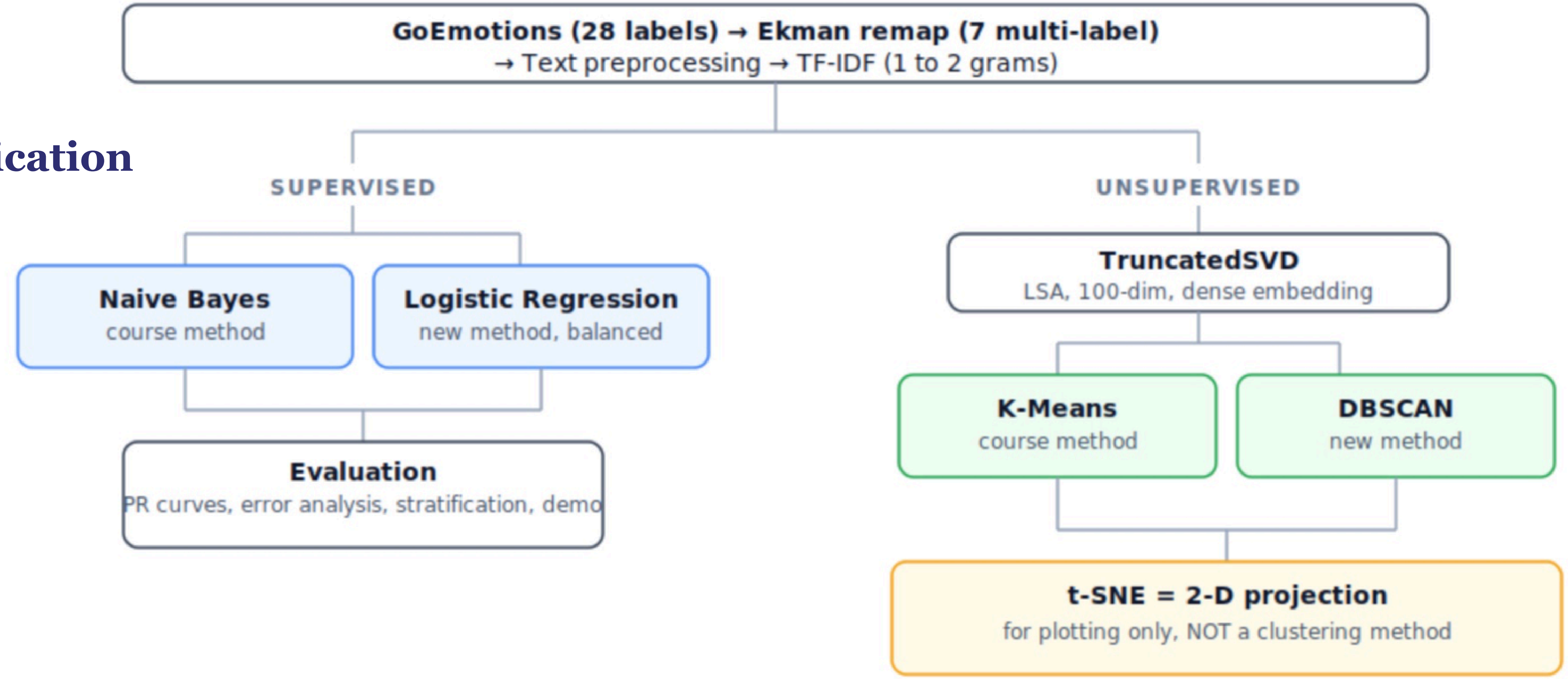
- K-means clustering
- DBSCAN
- t-SNE visualization

03 Supervised: Multi-label Classification

- Naive Bayes
- Logistic Regression

04 Evaluation

- F1 score
- Error Analysis



Dataset: GoEmotions

- **Source:** Google Research GoEmotions dataset

  Data type: Reddit comments

- **Task:** Emotion detection from text

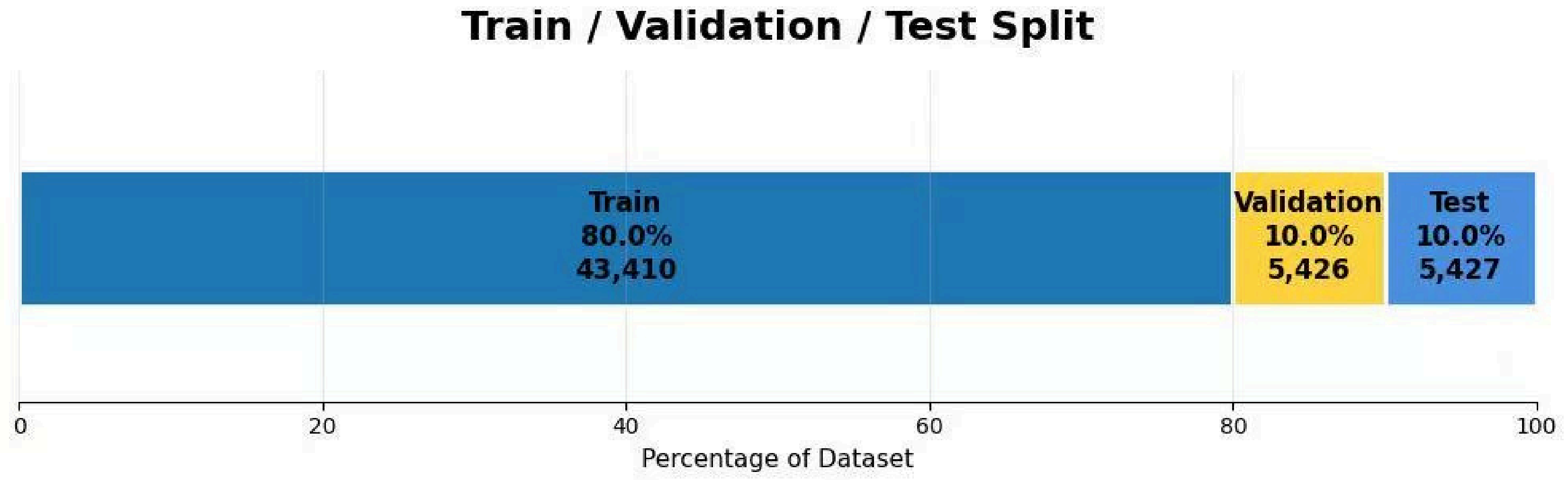
- **Original labels:** 28 fine-grained emotions

  Our project: group them into 7 emotion categories using eman mapping

text,admiration,amusement,anger,annoyance,approval,caring,confusion,curiosity,desire,disappointment,disapproval,disgust,embarrassment,excitement,fear,gratitude,grief,joy,love,nervousness,optimism,pride,realization,relief,remorse,sadness,surprise,neutral
That game hurt.,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0
Y'all really out here supporting authoritarian Venezuelan dictators huh.,0,0,0,0,0,0,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0

```
ekman_mapping = {
   "anger":   ["anger", "annoyance", "disapproval"],
   "disgust": ["disgust"],
   "fear":    ["fear", "nervousness"],
   "joy":     ["joy", "amusement", "approval", "excitement", "gratitude",
              "love", "optimism", "relief", "pride", "admiration",
              "desire", "caring"],
   "sadness": ["sadness", "disappointment", "embarrassment", "grief", "remorse"],
   "surprise": ["surprise", "realization", "confusion", "curiosity"],
   "neutral": ["neutral"]
}
```

Data Cleaning

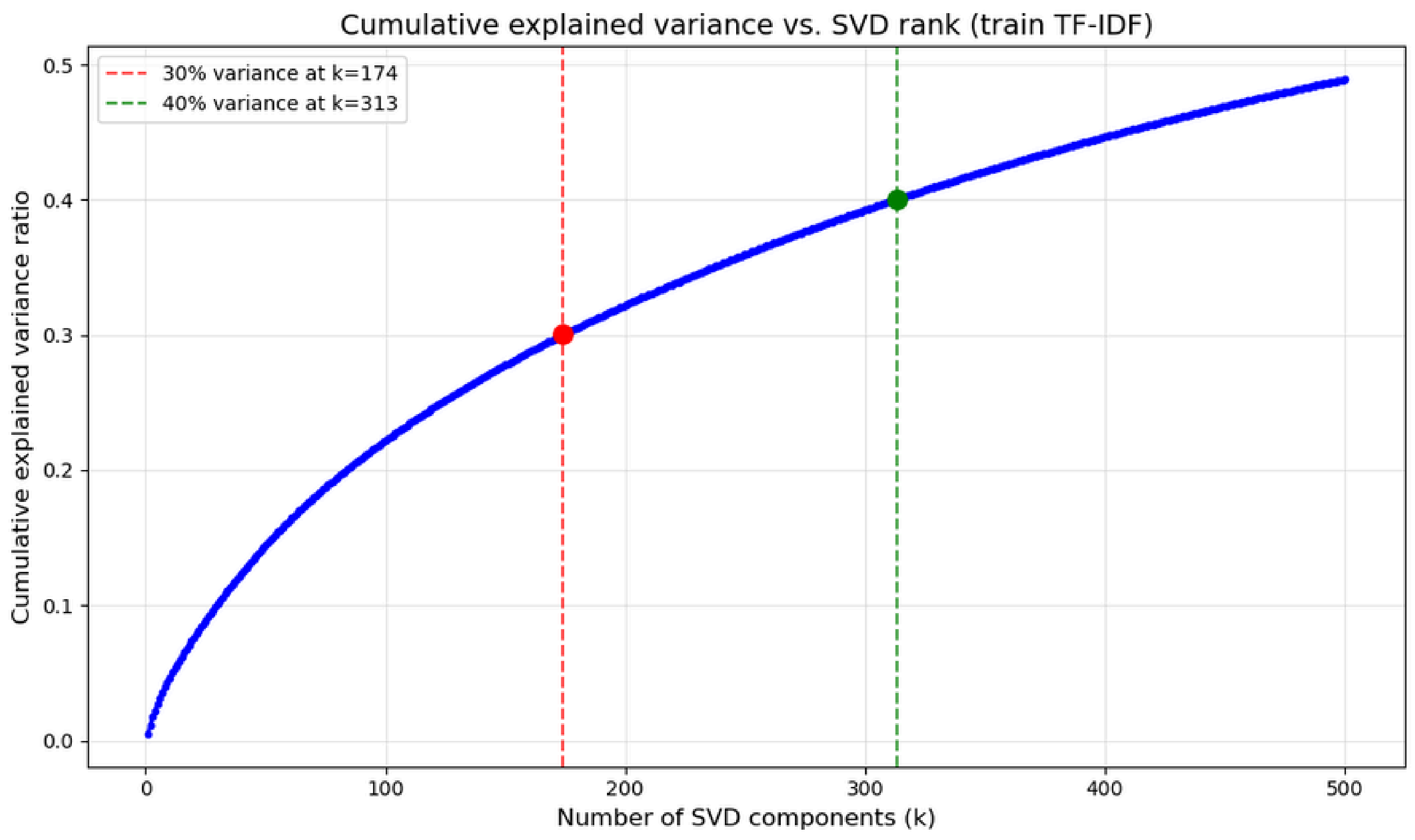


We prepared the data by:

- Keeping text and emotion labels
- Splitting into train, validation, and test sets
- Cleaning text:
  - lowercase
  - remove URLs
  - remove special
  - characters/numbers
  - remove stopwords
  - lemmatization

Turning Text to Vector

Pipeline: Raw text → Cleaned text → TF-IDF → Truncated SVD



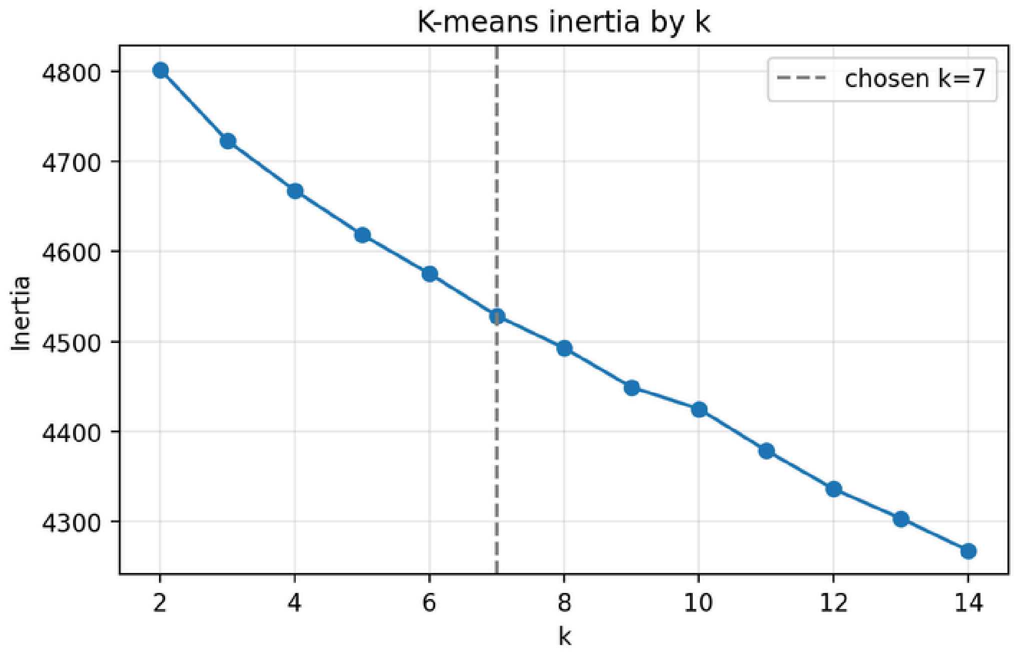
- TF-IDF: converts words into numerical vectors
- Truncated SVD: reduces high-dimensional vectors
  - 174 dimensions: least to reach 30% variance

K-means: find groups of similar comments without using the emotion labels

Objective: See if natural clusters in the text data would correspond to the 7 emotion categories

TF-IDF vocabulary size is 5000, truncated SVD reduced it to 100

Elbow point (inertia) and silhouette score to find opimized K from 2-14



Results

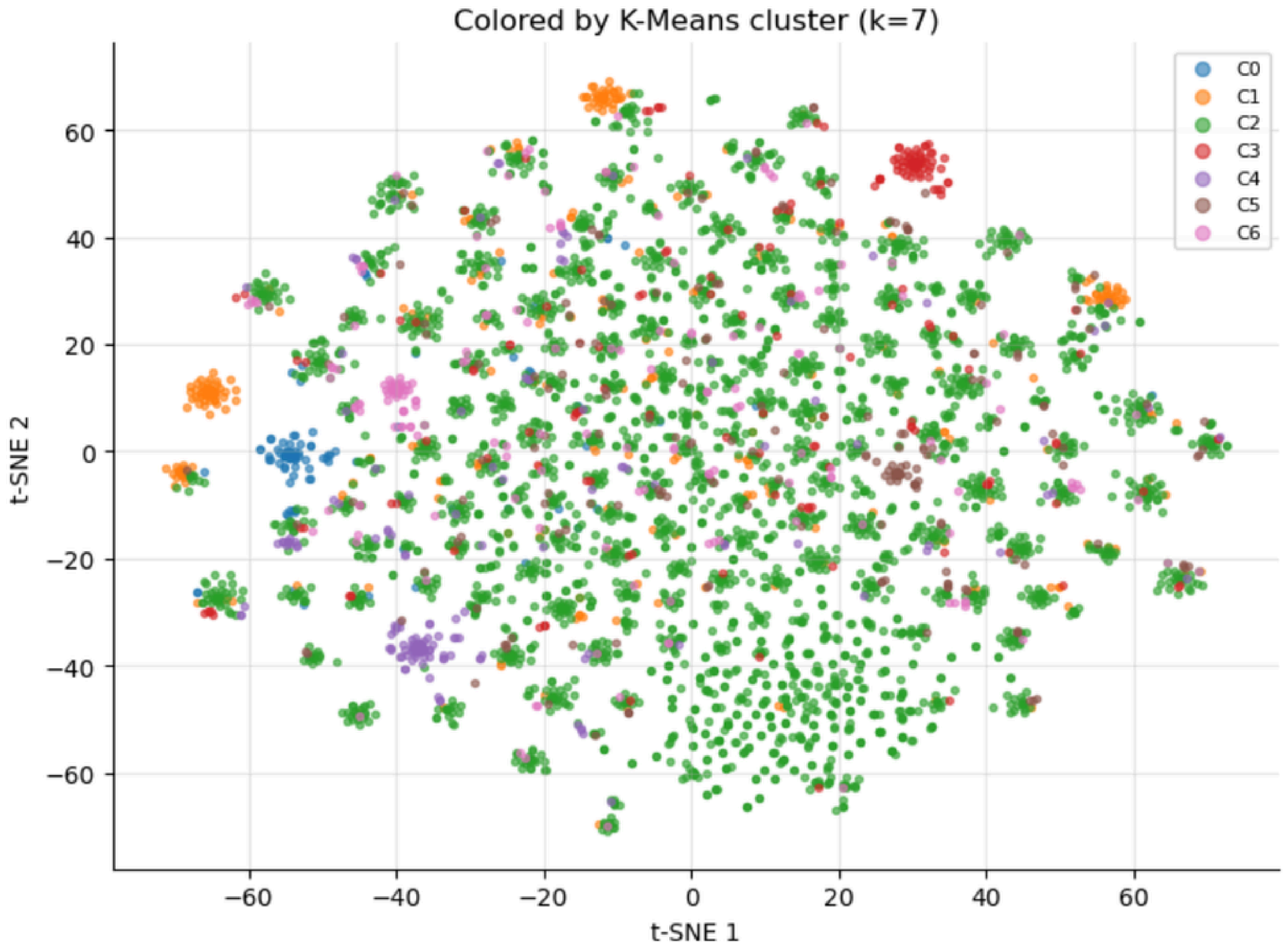
- K-selection diagnostic prefers a larger k:
- silhouette keeps rising up to k=14, where it reaches 0.0579.
- Final clustering uses k=7 because there are 7 Ekman labels

Why are silhouette scores so low?

- Emotion text is naturally overlapping (it was meant to be multi-labeled).
- The dataset is imbalanced.

Clustering

finding groups of similar comments without using the emotion labels



Data Visualization

We sampled 5000 comments and created two plots: one colored by K-means clusters and one colored by the true dominant emotion labels.

Results

- Some small local groups appeared
- Emotion colors were heavily mixed
- No clear separated emotion regions

Multinomial NB

One-vs-Rest:

We train seven separate binary models--one per emotion. Each model answers a simple question: for this post, is this specific emotion present or not? At prediction time we stack the seven binary decisions into a seven-bit vector.

moothing parameter Alpha

- **smaller**(trust more on data)
- **larger**(smoother estimation)

alpha_candidates = [1e-5,1e-4,1e-3, 1e-2, 0.1, 1.0, 10.0]

Compare Macro-F1 on val

Validation macro-F1 by MultinomialNB alpha :

```
 alpha=  1e-05  macro-F1=0.2491
 alpha= 0.0001  macro-F1=0.2491
 alpha=  0.001  macro-F1=0.2519  <-- selected
 alpha=   0.01  macro-F1=0.2519
 alpha=    0.1  macro-F1=0.2485
 alpha=    1.0  macro-F1=0.1900
 alpha=   10.0  macro-F1=0.0963
```

Logistic Regression

Regularization parameter C

- **smaller C** (stronger regularization, simpler model)
- **larger C** (fits data more closely, risk of overfitting)
- class_weight="balanced" (compensates for class imbalance)

```
C_candidates = [0.1, 0.5, 1.0, 2.0, 5.0]
```

One-vs-Rest:

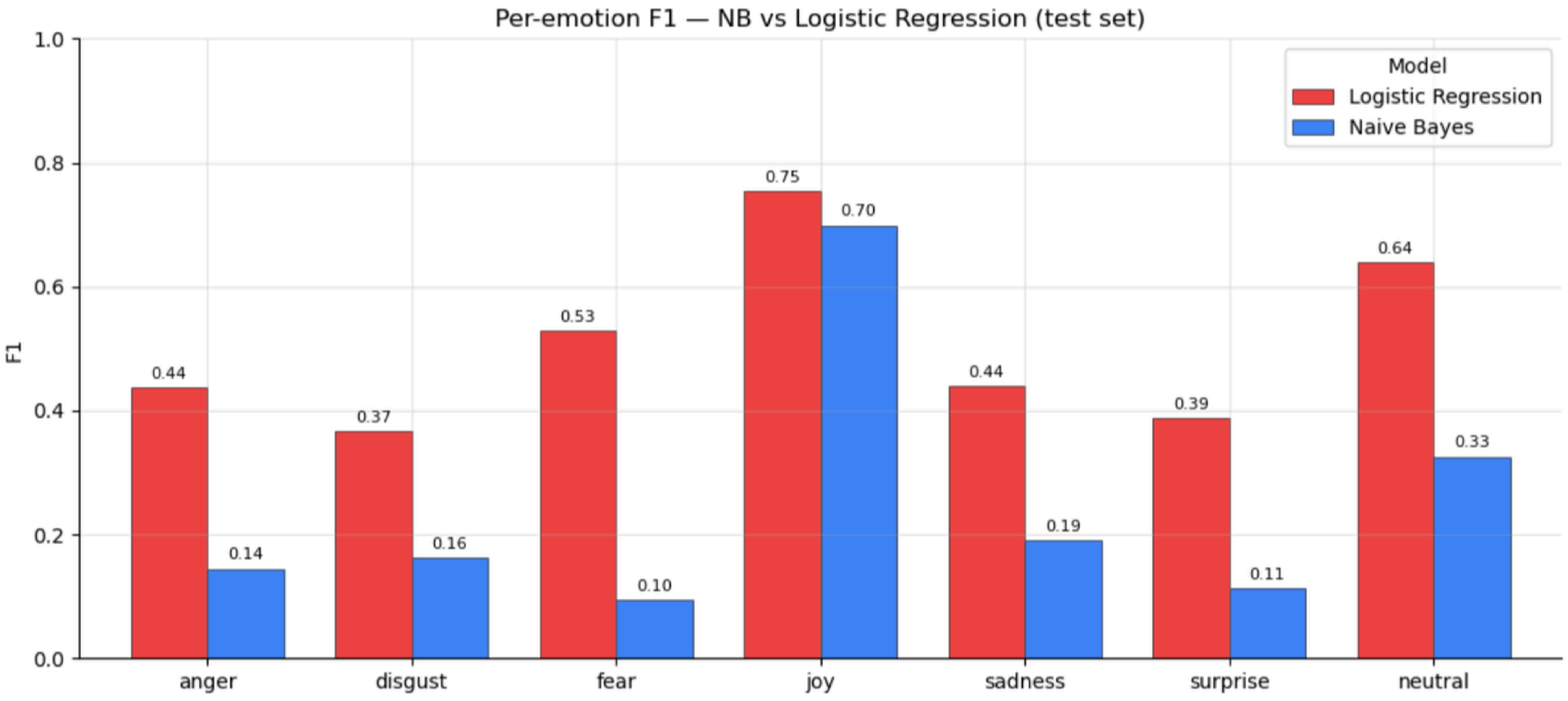
Trains 7 separate binary models, one per emotion. Uses a weighted sum of TF-IDF features pushed through a sigmoid to give independent probabilities.

Compare Macro-F1 on val

```
Validation macro-F1 by C:
  C=  0.1  macro-F1=0.4912  <-- selected
  C=  0.5  macro-F1=0.4831
  C=  1.0  macro-F1=0.4788
  C=  2.0  macro-F1=0.4712
  C=  5.0  macro-F1=0.4631

Best C: 0.1
```

Results:

Logistic Regression nearly doubles macro-F1 over the Naive Bayes baseline



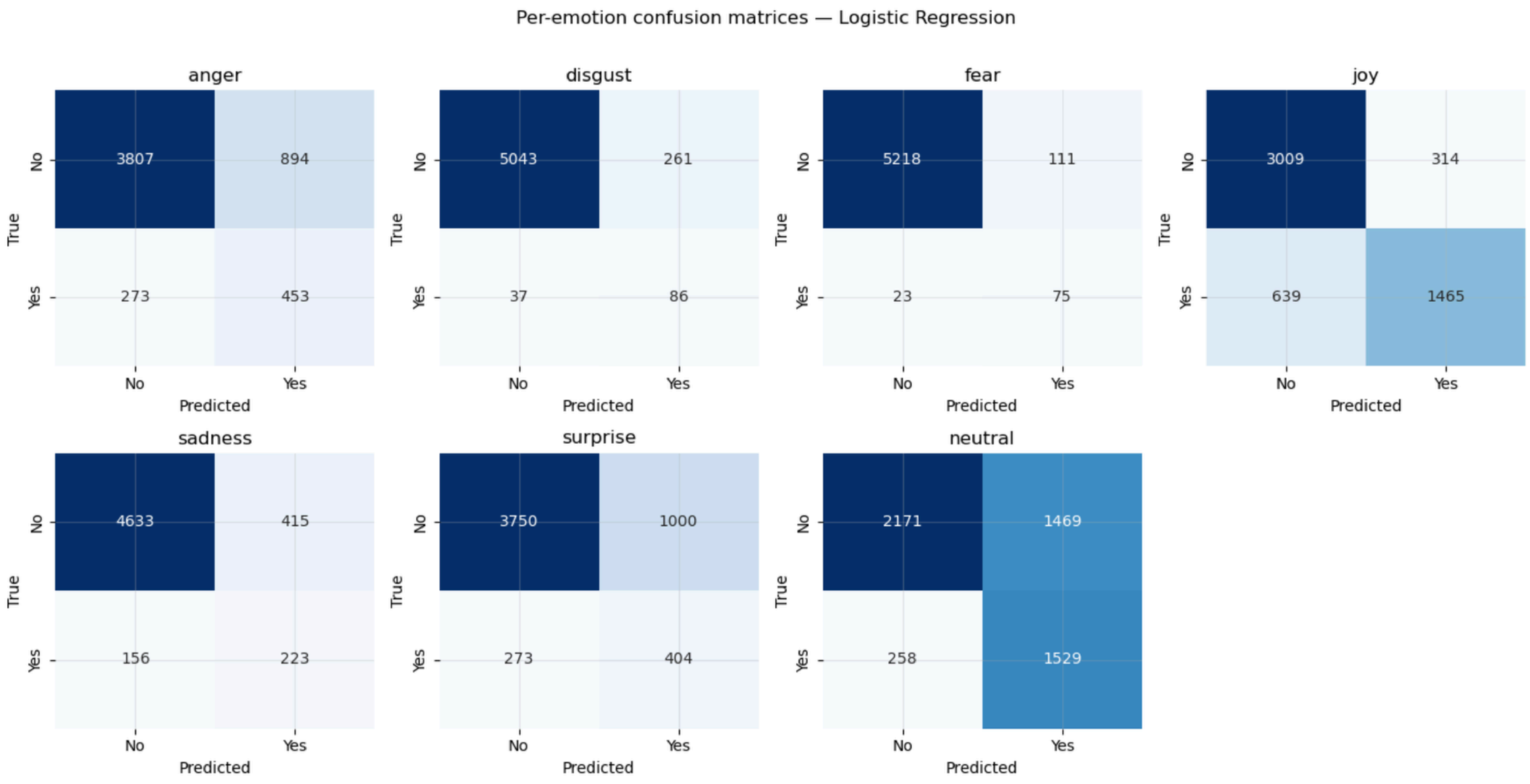
NB macro-F1 - 0.25
LogReg macro-F1 - 0.49

Results:

Model	Naive Bayes	Logistic Regression
Subset Accuracy	0.3072	0.3715
Hamming Loss	0.1226	0.1612
Macro Precision	0.7675	0.4226
Macro Recall	0.1742	0.6894
Macro F1	0.2472	0.5074
Weighted F1	0.3962	0.6062
Macro ROC-AUC	0.7505	0.8406

```
======================================================================
NAIVE BAYES
======================================================================
              precision    recall  f1-score   support

       anger       0.78      0.08      0.14       726
     disgust       0.92      0.09      0.16       123
        fear       0.71      0.05      0.10        98
         joy       0.83      0.60      0.70      2104
     sadness       0.79      0.11      0.19       379
    surprise       0.76      0.06      0.11       677
     neutral       0.58      0.23      0.33      1787

   micro avg       0.75      0.31      0.44      5894
   macro avg       0.77      0.17      0.25      5894
weighted avg       0.74      0.31      0.40      5894
 samples avg       0.34      0.32      0.33      5894

======================================================================
LOGISTIC REGRESSION
======================================================================
              precision    recall  f1-score   support

       anger       0.34      0.62      0.44       726
     disgust       0.25      0.70      0.37       123
        fear       0.40      0.77      0.53        98
         joy       0.82      0.70      0.75      2104
     sadness       0.35      0.59      0.44       379
    surprise       0.29      0.60      0.39       677
     neutral       0.51      0.86      0.64      1787

   micro avg       0.49      0.72      0.58      5894
   macro avg       0.42      0.69      0.51      5894
weighted avg       0.56      0.72      0.61      5894
 samples avg       0.56      0.73      0.61      5894
```

Results:


Per-emotion confusion matrices — Logistic Regression
anger
3807
894
273
453
disgust
5043
261
37
86
fear
5218
111
23
75
joy
3009
314
639
1465
sadness
4633
415
156
223
surprise
3750
1000
273
404
neutral
2171
1469
258
1529
True
No
Yes
Predicted

live demo

[37]:
```python
# === Example 2: strong negative emotion ===
INPUT_TEXT = "This is absolutely disgusting and I can't believe it happened."
explain_prediction(INPUT_TEXT)
```

```
======================================================================
INPUT:   This is absolutely disgusting and I can't believe it happened.
CLEANED: absolutely disgusting cant believe happened
======================================================================

Emotion      Probability  Predicted
----------------------------------------
disgust      0.9669       ✓
surprise     0.6445       ✓
anger        0.4051
sadness      0.3785
joy          0.2621
fear         0.2554
neutral      0.2379

======================================================================
WORD CONTRIBUTIONS per predicted emotion
======================================================================

  DISGUST:
    disgusting          +4.151
    absolutely          +0.708
    happened            -0.211
    believe             -0.282

  SURPRISE:
    happened            +0.733
    believe             +0.591
    disgusting          -0.252
    absolutely          -0.333
```

```
✓ TEXT:      I am so happy and excited about this!
  EXPECTED: ['joy']
  PRED:      ['joy']

✓ TEXT:      This is absolutely disgusting and I hate it.
  EXPECTED: ['disgust', 'anger']
  PRED:      ['anger', 'disgust']

✓ TEXT:      I feel nervous and scared about what will happen.
  EXPECTED: ['fear']
  PRED:      ['fear']

✓ TEXT:      I miss them so much, my heart is broken.
  EXPECTED: ['sadness']
  PRED:      ['sadness', 'neutral']

✓ TEXT:      Wait, what?! I can't believe this happened.
  EXPECTED: ['surprise']
  PRED:      ['surprise']

✓ TEXT:      The weather is fine today.
  EXPECTED: ['neutral']
  PRED:      ['neutral']

✗ TEXT:      Oh great, another wonderful Monday morning.
  EXPECTED: ['sadness', 'anger']
  PRED:      ['joy']
```

Where the model fails

Misclassifications into four interpretable patterns

Short text

≤4 words. Not enough lexical signal.

Example

"Lol" → predicted neutral, truly joy

Negation

TF-IDF treats words independently.

Example

"I'm not unhappy" → predicted sadness

Sarcasm

Tone is in pragmatics, not words.

Example

"Oh great, another Monday" → joy

Ambiguous multi-label

Mixed emotions; model picks the strongest.

Example

"Furious but relieved" → only anger

Conclusions

What worked

- Macro-F1 doubled vs baseline (0.25 → 0.49)
- Joy reliably detected (F1 0.77)
- Word-level interpretability for every prediction
- Two unsupervised methods agree

Known limitations

- TF-IDF cannot capture negation or sarcasm
- Rare emotions (disgust, fear) still hard
- Threshold fixed at 0.5 for every emotion
- Short comments perform worst across all emotions

Thank You!

DM1590 - Machine Learning for Media Technology

Group 15: Fuad Mammadov, Qingya Li, Xinyutong Zhang, Jintong Jiang